A

Mini Project Report

on

AgroAssist

Submitted in partial fulfillment of the requirements for the degree

Second Year Engineering – Computer Science Engineering (Data Science)

by

Piyush Sase - Moodle ID: 24107056

Satyam Patel - Moodle ID:24107105

Harsh Sarwankar - Moodle ID:24107014

Pushkar Patole – Moodle ID:24107137

Under the guidance of

Ms. Swati Parhad

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (DATA SCIENCE)

A.P. SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W)-400615

UNIVERSITY OF MUMBAI

Academic year: 2025-26

CERTIFICATE

This is to certify that the Mini Project report on "AgroAssist" has been submitted by **Piyush Sase (Student ID: 24107056), Satyam Patel (Student ID:24107105), Harsh Sarwankar (Student ID: 24107014) and Pushkar Patole (Student ID: 24107137)** who are bonafide students of A. P. Shah Institute of Technology, Thane as a partial fulfillment of the requirement for the degree in Computer Science Engineering (Data Science), during the academic year 2025-2026 in the satisfactory manner as per the curriculum laid down by University of Mumbai.

Prof. Swati Parhad
Guide

Dr. Pravin Adivarekar
HOD, CSE (Data Science)

Dr. Uttam D. Kolekar
Principal

External Examiner:

1. _____

Internal Examiner:

1. _____

Place: A. P. Shah Institute of Technology, Thane
Date:

ACKNOWLEDGEMENT

This project would not have come to fruition without the invaluable help of our guide, Prof. Swati Parhad. We express our sincere gratitude towards our HoD, Dr. Pravin Adivarekar, and the Department of Computer Science Engineering (Data Science) for providing us with the opportunity as well as the support required to pursue this project.

We would also like to thank our project coordinators Prof.Hardiki Patil and Prof. Dipali Gat who gave us their valuable suggestions and ideas when we were in need of them. Their timely guidance helped us steer the project in the right direction.

We would also like to thank our peers and all the faculty members of the Department of Computer Science Engineering (Data Science) for their helpful suggestions and continued motivation throughout the development of this project.

TABLE OF CONTENTS

Chapter 1 Introduction	1
1.1 Purpose	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope	3
Chapter 2 Proposed System	4
2.1 Overview	4
2.2 Block Diagram	4
2.3 Features and Functionality	5
Chapter 3 Project Outcomes	6
Chapter 4 Software Requirements	7
Chapter 5 Project Design	8
5.1 System Architecture	8
5.2 Workflow Diagrams	9
Chapter 6 Project Scheduling	10
Chapter 7 Results	11
Chapter 8 Conclusion	17
References	18

Chapter 1

Introduction

Agriculture is the backbone of the Indian economy, employing a significant portion of the population and contributing substantially to the national GDP. Despite its importance, the agricultural sector faces persistent challenges such as lack of timely information, inefficient task management, and limited access to weather advisories. AgroAssist is a full-stack agriculture management platform developed to address these challenges by providing farmers, administrators, and agricultural stakeholders with a unified digital solution for managing crop records, farmer profiles, farm tasks, and weather alerts in an efficient and accessible manner.

1.1 Purpose

The purpose of this document is to present a comprehensive mini project report for AgroAssist, a web and mobile agriculture management application built using Django REST Framework on the backend and Flutter on the frontend. The report is intended for the academic review panel, project guides, and faculty coordinators of A. P. Shah Institute of Technology. It details the problem statement, system design, technology stack, and outcomes of the project developed during the academic year 2025-26.

1.2 Problem Statement

Small and marginal farmers in India often lack access to structured tools for managing their agricultural activities. Information about crop cultivation, task scheduling, and weather conditions is scattered across multiple sources and is often inaccessible on mobile devices. There is no single integrated platform that allows farm administrators to manage farmer profiles, track task lifecycles, issue weather warnings, and provide crop recommendations in a role-aware manner. This gap results in poor productivity, resource wastage, and an inability to respond proactively to adverse weather conditions. AgroAssist aims to bridge this gap by providing a unified, role-based digital platform tailored to the needs of the agricultural sector.

1.3 Objectives

The primary objectives of the AgroAssist project are as follows:

- To develop a full-stack web and mobile application for agricultural management.

- To implement a REST API backend using Django and Django REST Framework for managing crops, farmers, tasks, and weather data.
- To build a cross-platform Flutter frontend that communicates with the backend via HTTP requests.
- To provide role-aware workflows that distinguish between admin and farmer access levels.
- To enable task lifecycle tracking with proper validation and status updates.
- To deliver weather alerts and summaries to help farmers take proactive decisions.
- To implement token-based authentication with session expiry handling for security.

1.4 Scope

The scope of AgroAssist includes the development of a full-stack monorepo system comprising a Django REST backend and a Flutter frontend. The system supports four primary domains: Crops, Farmers, Tasks, and Weather. The backend provides RESTful API endpoints for each domain, which the Flutter client consumes via HTTP requests. The application supports web deployment and Android emulator access. The scope is limited to the functionality listed above and does not include integration with government agricultural databases, real-time satellite imagery, or e-commerce features in the current version.

Chapter 2

Proposed System

AgroAssist is proposed as an integrated digital agriculture management platform that replaces manual, paper-based farm record keeping with a structured, role-aware web and mobile application. The proposed system leverages modern web technologies to provide real-time access to farm data, weather advisories, and task management capabilities. The system is designed with scalability and ease of use in mind, ensuring that both tech-savvy administrators and less experienced farmers can interact with the application effectively.

2.1 Overview of Proposed System

The proposed system is a client-server architecture consisting of a Django REST backend and a Flutter frontend. The backend exposes RESTful API endpoints for all farm operations, while the Flutter client provides a responsive user interface accessible via web browsers and Android devices. The system adopts a role-based access control model where administrators have full system access and farmers are restricted to their own profile and task data.

2.2 Block Diagram

The following block diagram illustrates the high-level architecture of the AgroAssist system, showing the interaction between the Flutter frontend client, the Django REST API backend, the database layer, and the external weather data source.

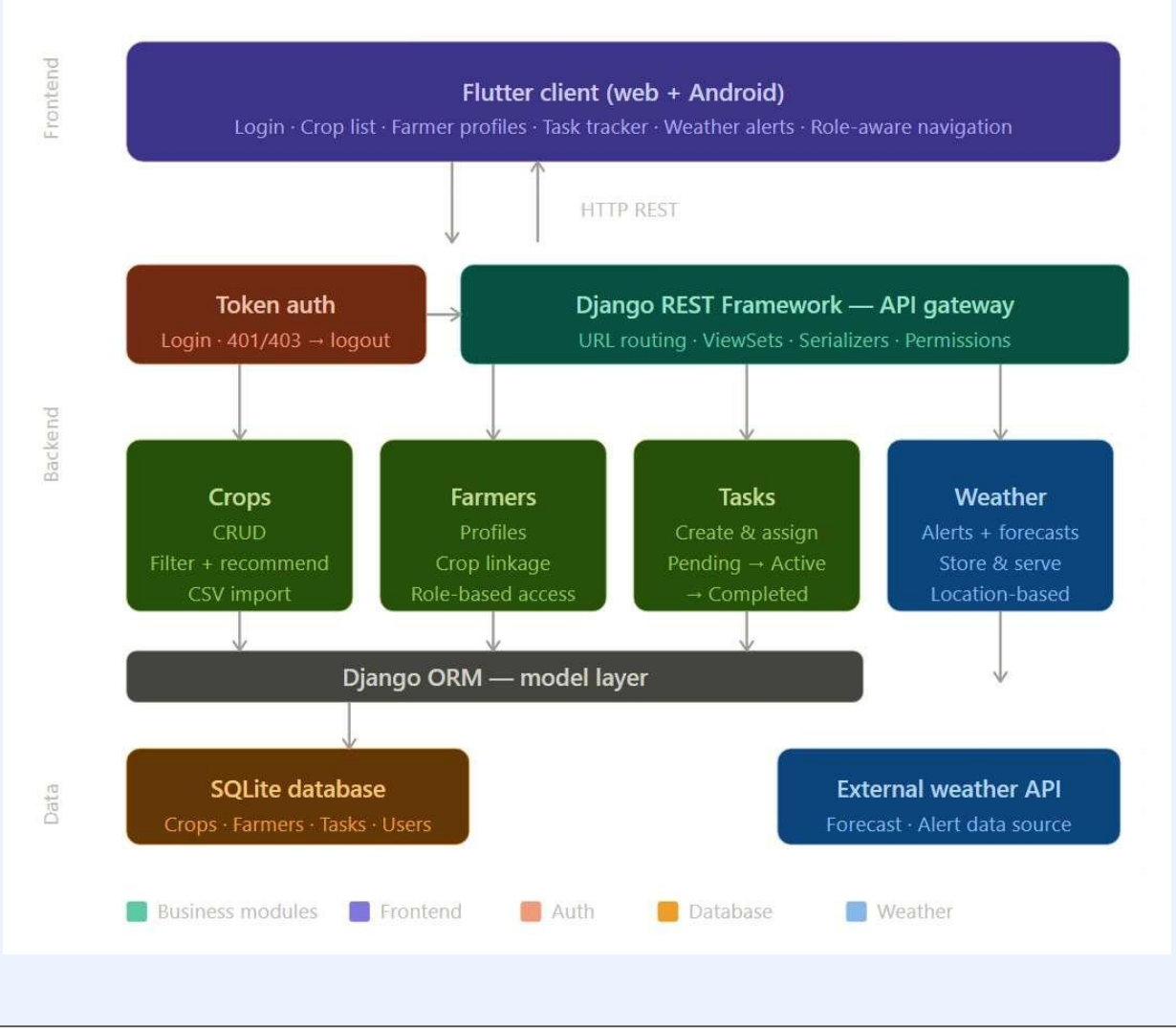


Figure 2.2: Block Diagram of AgroAssist Proposed System

As shown in Figure 2.2, the Flutter client sends HTTP requests to the Django REST API. The API processes requests through serializers and viewsets, and the ORM layer persists and retrieves data from the SQLite database. The weather module independently fetches and stores weather alert data, which is then served to the Flutter client on demand.

2.3 Features and Functionality

AgroAssist provides the following key features:

- Crop Management: CRUD operations on crop records with filtering and recommendation support.

- Farmer Profile Management: Registration, profile updates, and linking farmers to specific crops.
- Task Management: Task creation with lifecycle tracking (Pending → In Progress → Completed) and validation.
- Weather Alerts: Storage and delivery of weather advisories and forecasts to farmers.
- Token-Based Authentication: Secure login with session persistence in Flutter and forced logout on 401/403 responses.
- Role-Aware Access Control: Separate access levels for Admin and Farmer roles throughout the application.

Chapter 3

Project Outcomes

The successful implementation of AgroAssist is expected to produce the following tangible and measurable outcomes that contribute to the improvement of agricultural management practices and demonstrate the application of software engineering principles in a real-world context.

- A fully functional full-stack web and mobile application accessible via browser and Android devices.
- A well-structured Django REST API backend with modular apps for Crops, Farmers, Tasks, Weather, and Authentication.
- A Flutter frontend application with role-aware navigation, real-time API integration, and session management.
- A working task lifecycle management system that enforces status transitions and validation rules.
- A weather alert module that stores and displays location-based weather advisories to farmers.
- A secure token-based authentication system with automatic session expiry handling.
- A CSV bulk import facility for populating crop and farmer data efficiently.
- Improved understanding of REST API design, Flutter development, and full-stack integration among team members.

Upon completion, the project demonstrates the practical use of technologies such as Python, Django, Dart, and Flutter in building a scalable and deployable agricultural platform. The project also exhibits the team's ability to apply software engineering concepts including requirement analysis, system design, modular development, testing, and documentation.

Chapter 4

Software Requirements

This chapter outlines the hardware and software requirements for developing and running the AgroAssist platform. The system has been designed to work on standard development machines with minimal hardware requirements, making it accessible for academic development environments.

4.1 Software Requirements

Component	Specification / Version
Operating System	Windows 10/11 / Ubuntu 20.04 or later
Backend Language	Python 3.10+
Backend Framework	Django 4.x, Django REST Framework 3.x
Frontend Language	Dart 3.x
Frontend Framework	Flutter 3.x
Database	SQLite 3 (default), PostgreSQL (production)
IDE / Editor	VS Code / PyCharm / Android Studio
API Testing Tool	Postman / Thunder Client
Version Control	Git, GitHub
Deployment	Vercel (frontend), Local Django server
Browser	Google Chrome / Microsoft Edge (latest)

Table 4.1: Software Requirements

System Requirements (Short Description)

The application is developed using a modern technology stack to ensure efficiency and scalability. It supports operating systems like Windows 10, Windows 11, and Ubuntu 20.04 or later.

The backend is built with Python 3.10+ using Django and Django REST Framework, while the frontend uses Dart and Flutter.

For the database, SQLite is used during development and PostgreSQL for production.

Development is done using Visual Studio Code, PyCharm, or Android Studio, with API testing via Postman or Thunder Client.

Version control is managed using Git and GitHub. The frontend is deployed on Vercel, and the system runs on modern browsers like Google Chrome and Microsoft Edge.

Chapter 5

Project Design

The project design chapter presents the architectural and workflow design of the AgroAssist system. This includes the overall system architecture, the data flow between components, and the workflow diagrams for the primary use cases. Good system design is essential to ensure the application is maintainable, scalable, and aligned with stated requirements.

5.1 System Architecture

AgroAssist follows a client-server architecture pattern. The Flutter web/mobile application acts as the client that sends HTTP REST requests to the Django backend server. The backend processes requests through a layered architecture comprising URL routing, viewsets, serializers, and the Django ORM. The database layer uses SQLite for development and can be switched to PostgreSQL for production. The authentication layer uses token-based authentication to secure all protected API endpoints. a layered architecture comprising URL routing, viewsets, serializers, and the Django ORM. The database layer uses SQLite for development and can be switched to PostgreSQL for production. The authentication layer uses token-based authentication to secure all protected API endpoints.

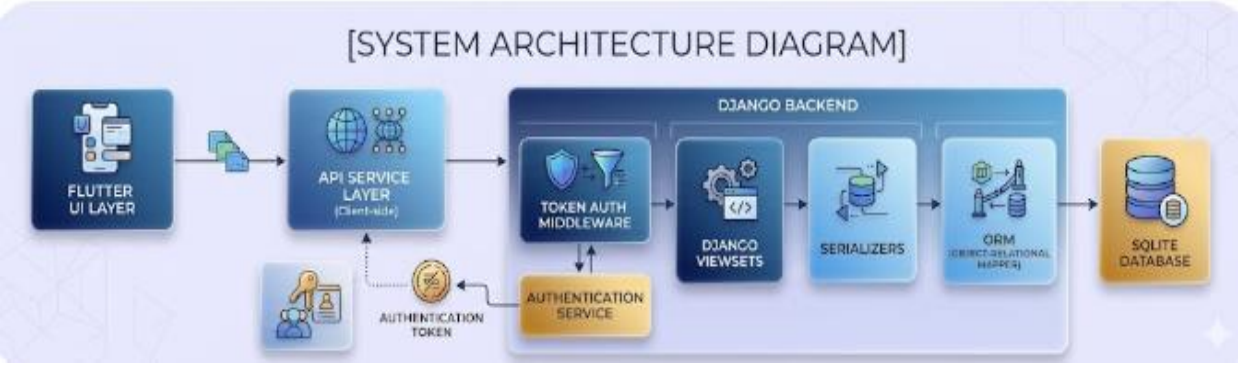


Figure 5.1: AgroAssist System Architecture Diagram

5.2 Workflow Diagrams

5.2.1 User Authentication Workflow

Figure 5.2 illustrates the authentication workflow of the AgroAssist system. When a user opens the application, the Flutter auth service checks for a locally stored token. If a valid token exists, the user is directed to the home screen. Otherwise, the user is redirected to the login screen. Upon entering valid credentials, the backend generates an authentication token which is stored locally. If the token expires or

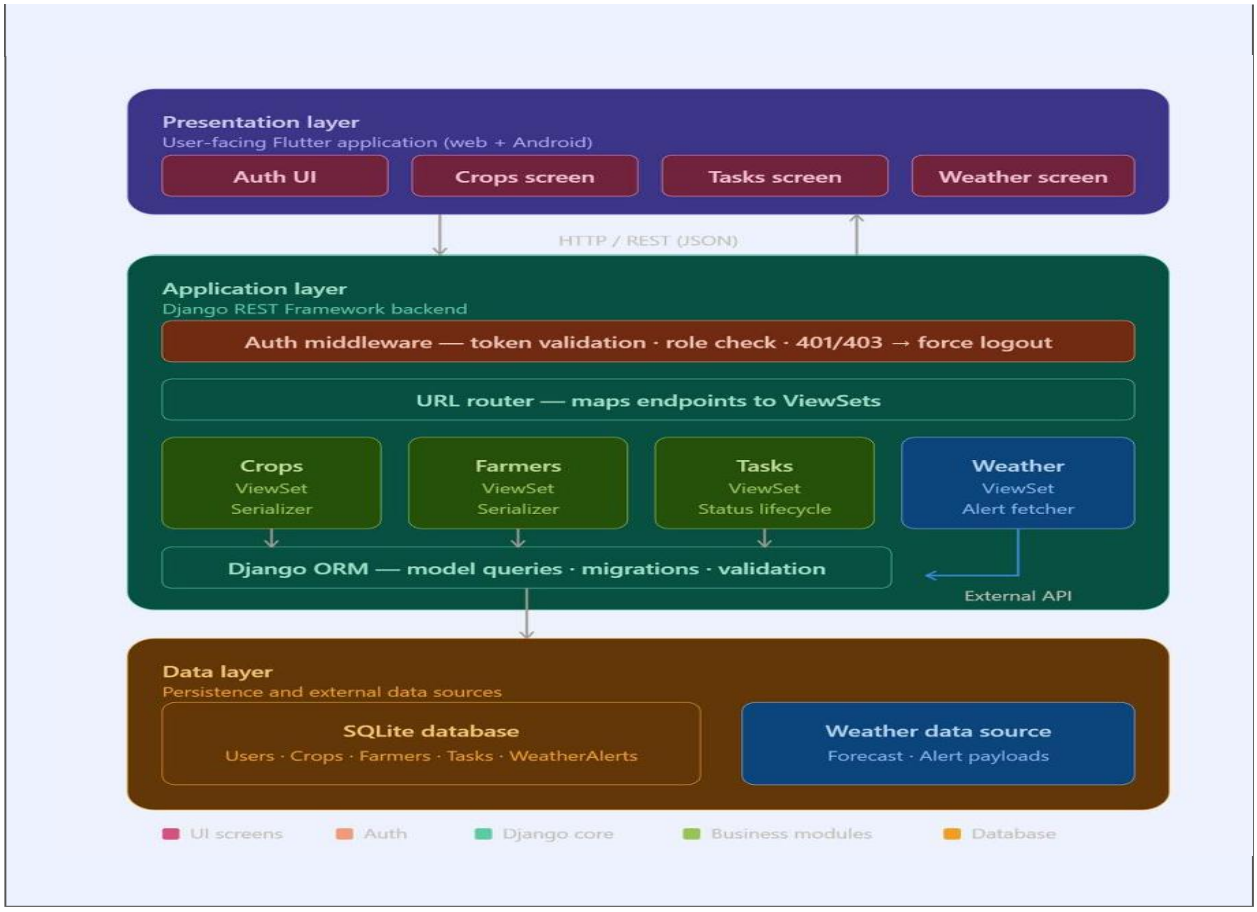


Figure 5.2: User Authentication Workflow Diagram

This diagram illustrates a classic **three-tier architecture** for the application, detailing how the user interface, business logic, and data storage interact.

Built using the **Flutter framework** for both Web and Android, this layer is responsible for the user experience. It consists of dedicated screens for **Authentication**, **Crop Management**, **Task Tracking**,

and **Weather updates**. It communicates with the backend exclusively via **HTTP/REST** calls, sending and receiving data in **JSON** format

5.2.2 Task Management Workflow

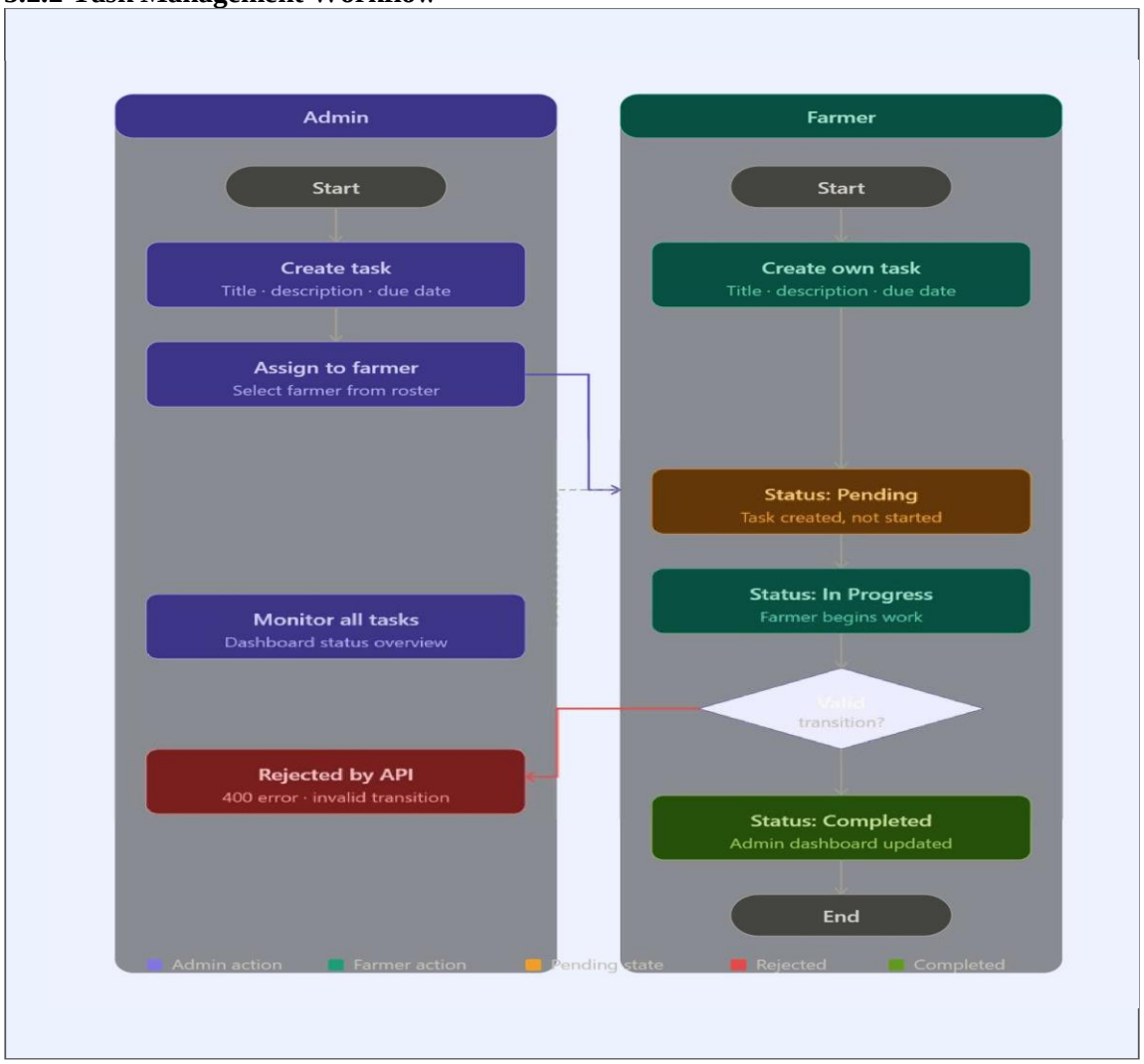


Figure 5.3: Task Management Workflow Diagram

5.2.3 Crop Recommendation Workflow

Figure 5.3 presents the detailed task lifecycle workflow embedded within the AgroAssist agricultural management system. This workflow is designed to manage operational activities from a central administrative level down to the field, ensuring clarity, accountability, and real-time visibility. The system adopts a structured, automated approach to guide each task from its inception to final verification and completion. Let’s dive deep into the specific mechanisms of each phase, the user interactions, the system’s backend logic, and the strategic oversight available to managers.

The entire process begins with a central administrator, a manager, or a supervisor logged into the AgroAssist Management Dashboard on a desktop application. They create a new task through a comprehensive form that requires a title (e.g., "Apply Fertilizer on Field 4"), a detailed description, a precise location (GPS coordinates or polygon drawn on an integrated

map), a deadline, and a priority level. The administrator has the option to attach supportive documents like product manuals, safety instructions, or a photo of the starting condition. A task can be assigned to a single field worker, known as a farmer, or even a team, based on their expertise and current workload which is visible to the admin.

An important initial step, often overlooked, is the 'Draft' state. The admin can save a task as a draft to refine the details, check resource availability, or coordinate with multiple parties before it’s made public. Once finalized, the admin clicks a 'Publish and Assign' button. At this moment, the task's journey truly begins, and it enters the **'Pending'** state. This transition automatically triggers an event in the backend. A "New Task Assigned" push notification is instantly sent to the farmer's mobile device via the AgroAssist mobile app, and a summary email is dispatched. This notification includes a brief overview to ensure it catches their attention immediately.

On the farmer's side, they access the "My Tasks" screen on their mobile app, which presents a list of all assigned work, sorted by a combination of priority and deadline. New tasks are clearly flagged. When the farmer reviews the details and is prepared to begin, they tap a "Start Task" button. This action is a pivotal transition. The backend server receives this request and, before any update, performs a validation check. This check, as part of the state machine pattern, ensures that a task cannot jump from 'Draft' to 'In Progress' directly, or from 'Completed' back to 'In Progress'. If it is in the valid 'Pending' state, the system records the precise start timestamp, updates the status in the main database to **'In Progress'**, and makes a corresponding update to the admin dashboard. Some versions of AgroAssist might enforce a "Proof of Start" by requiring a farmer to upload a photo of prepared equipment or check-in at the specific location to activate the task.

5.3 describes the crop recommendation and management workflow. A farmer or admin inputs query parameters such as soil type and season. The backend crop module filters available crop records and returns suitable recommendations. The farmer can then create a crop entry linked to their profile for

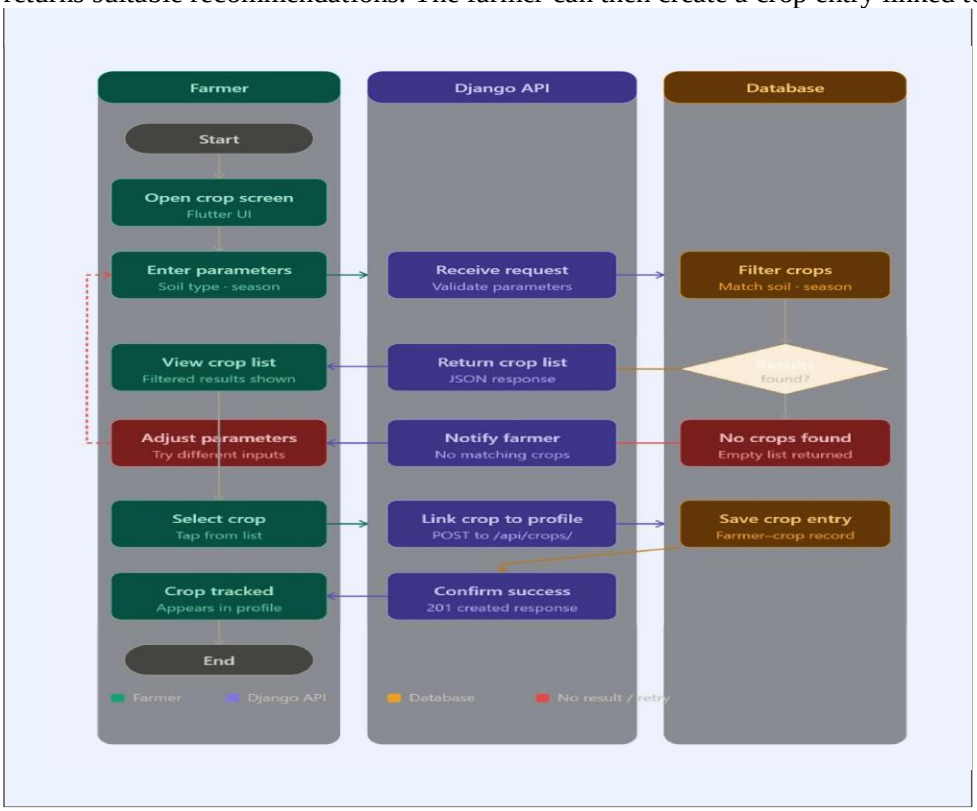


Figure 5.4: Crop Recommendation Workflow Diagram

1. Input and Validation (Farmer & Django API)

The process begins in the Farmer lane (Flutter UI), where the user enters specific environmental parameters like soil type and season. This data is sent to the Django API, which acts as the logic layer to validate the parameters before querying the data.

2. Data Retrieval and Conditional Logic (Database)

The Database filters the available crop records to find matches for the farmer's input.

- Success Path: If results are found, the database returns them to the API, which sends a JSON response back to the farmer to display a "Filtered results" list.
- Failure Path: If no matches exist, the system triggers a "No crops found" notification, prompting the farmer to return to the beginning and adjust their parameters.

3. Action and Persistence

Once the farmer selects a specific crop from the list, the frontend sends a POST request to the /api/crops/ endpoint. The Django API processes this request to link that crop to the farmer's unique profile, and the Database saves the record.

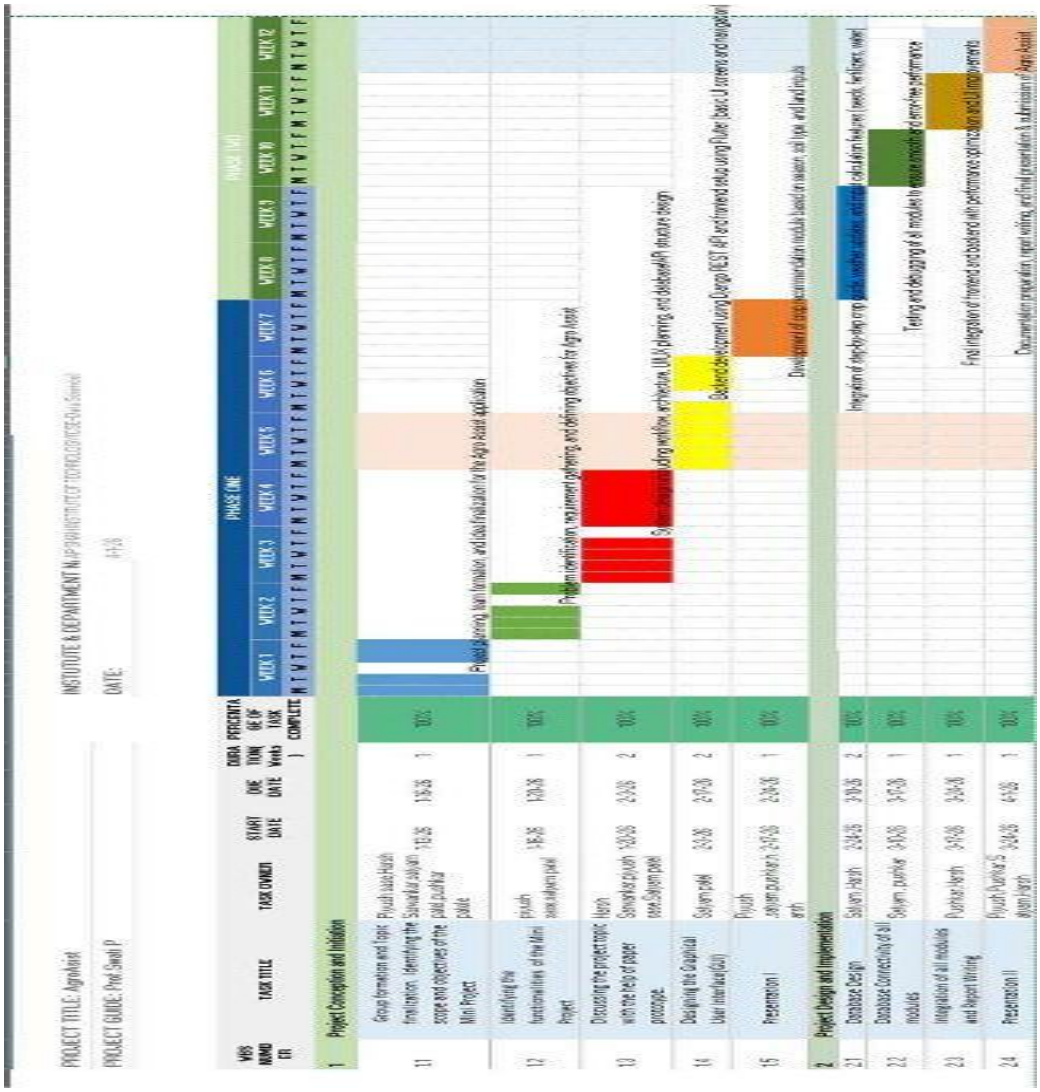
4. Confirmation

Finally, the backend returns a 201 Created status code to confirm the entry was successful, and the Flutter UI updates to show the crop is now being tracked in the farmer's profile.

Chapter 6

Project Scheduling

Project scheduling is essential for ensuring that all project activities are completed within the defined time frame. The Gantt chart below presents the planned schedule for the AgroAssist project covering the months of July to October 2025. Each task is mapped to the month(s) during which it was planned to be executed.



Chapter 7

Results

This chapter presents the screenshots of the AgroAssist application demonstrating the working of all major modules. Each figure is labelled with a reference number and described to explain the functionality visible in the screenshot. The screenshots below capture the application running in a web browser environment.

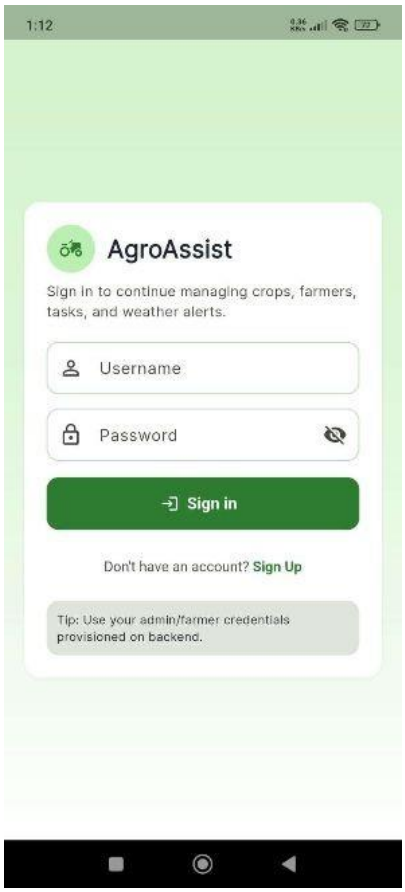


Figure 7.1: Login Screen

Figure 7.1 captures the initial gateway to the AgroAssist ecosystem: the Login Screen. This interface is designed with a "mobile-first" philosophy, ensuring that both field workers and office-bound administrators can gain access with minimal friction. The UI prompts for standard credentials—a unique username and a secure password—while offering a "Forgot Password" link to initiate a recovery workflow. Behind the scenes, the Flutter frontend communicates with the Django REST API using an encrypted POST request.

Upon a successful match in the database, the backend generates a secure JWT (JSON Web Token). This token is then passed back to the mobile device and stored securely using local storage (like Flutter Secure Storage), eliminating the need for the user to re-login every time they open the app. Crucially, this screen acts as the primary "router" for the application; the backend

payload includes the user’s specific role (e.g., 'Admin', 'Farmer', or 'Supervisor'). Based on this metadata, the application logic executes a conditional redirection, landing the user on a dashboard customized to their specific permissions and operational needs.

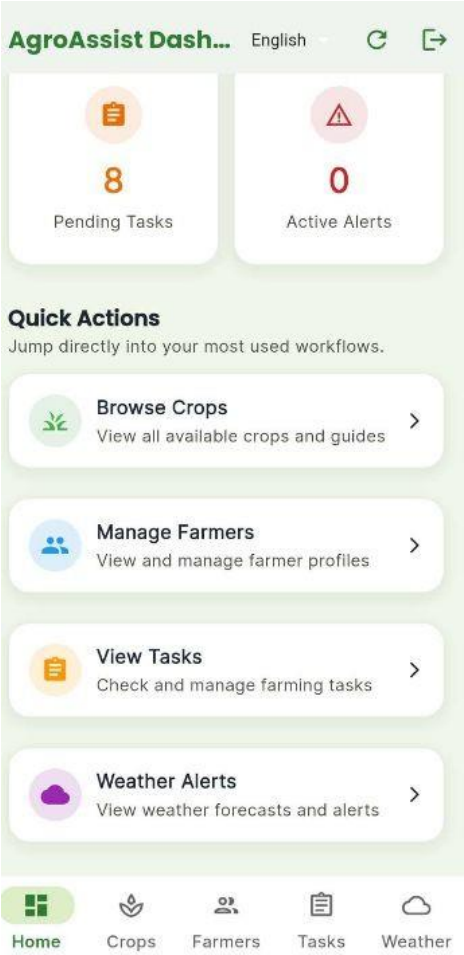


Figure 7.2: Admin Dashboard

Figure 7.2 illustrates the Admin Dashboard, the strategic hub that provides a comprehensive "360-degree" view of the entire farming operation. This screen is engineered to prevent information overload by utilizing a card-based layout that summarizes high-level data into digestible KPIs (Key Performance Indicators). The primary navigation menu offers one-tap access to four critical domains: Crops, Farmers, Tasks, and Weather.

At a glance, an administrator can see "Summary Statistics," such as the total acreage currently under cultivation, the number of farmers currently active in the field, and the count of critical tasks approaching their deadlines. These widgets often utilize color-coded indicators—green for optimal status and red for urgent attention—allowing for "management by exception." This centralized dashboard minimizes the time spent digging through individual records, ensuring that the farm manager can focus on high-level strategy and resource allocation while remaining tethered to the real-time heartbeat of the farm's daily activities.

an administrator can see "Summary Statistics," such as the total acreage currently under cultivation, the number of farmers currently active in the field, and the count of critical tasks approaching their deadlines. These widgets often utilize color-coded indicators—green for optimal status and red for urgent attention—allowing for "management by exception." This

centralized dashboard minimizes the time spent digging through individual records, ensuring that the farm manager can focus on high-level strategy and resource allocation while remaining tethered to the real-time heartbeat of the farm's

layout that summarizes high-level data into digestible KPIs (Key Performance Indicators). The primary navigation menu offers one-tap access to four critical domains: Crops, Farmers, Tasks, and Weather.

At a glance, an administrator can see "Summary Statistics," such as the total acreage currently under cultivation, the number of farmers currently active in the field, and the count of critical tasks approaching their deadlines. These widgets often utilize color-coded indicators—green for optimal status and red for urgent attention—allowing for "management by exception." This centralized dashboard minimizes the time spent digging through individual records, ensuring that the farm manager

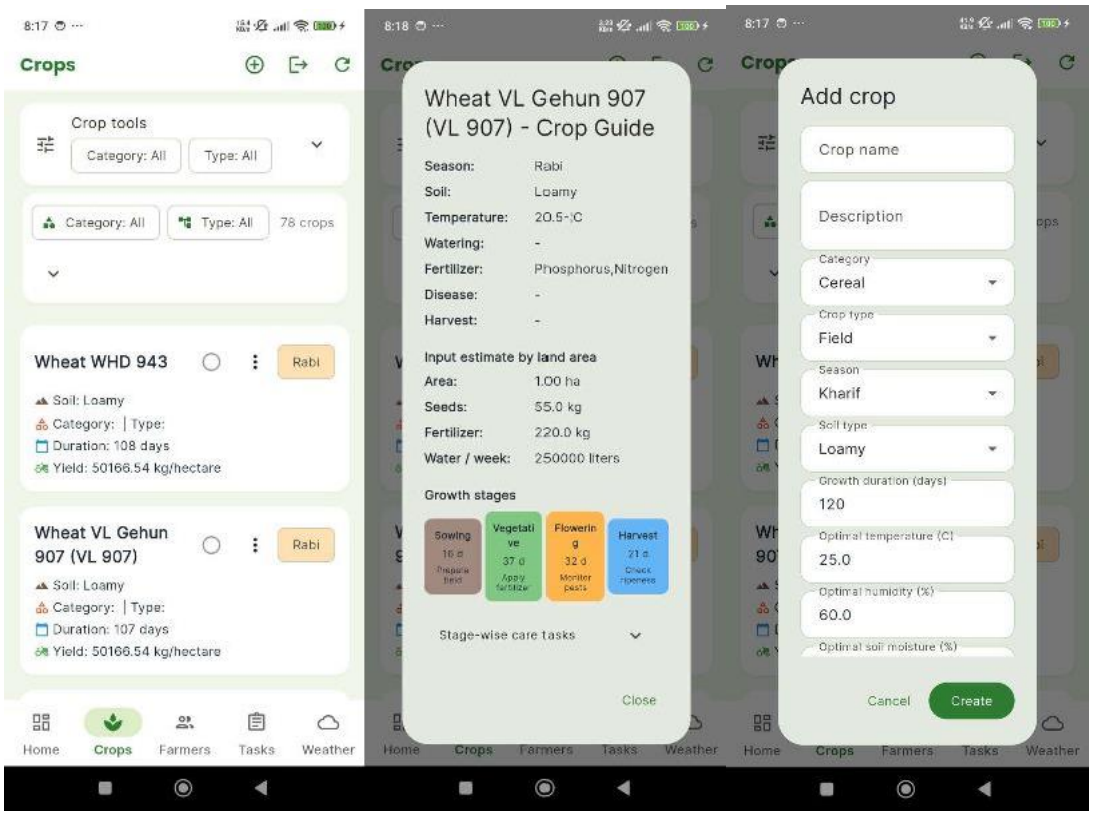


Figure 7.3: Crop Management Screen

Figure 7.3 displays the Crop Management screen where the admin or authorized user can view, add, update, and delete crop records. The list supports filtering by crop type and season, enabling efficient crop recommendation and management.displays the **Crop Management** screen, a robust database interface designed for the lifecycle tracking of agricultural assets. This module empowers administrators to perform full **CRUD (Create, Read, Update, Delete)** operations on crop records. Each entry includes essential data points such as crop variety, planting date, expected harvest window, and the specific plot or field ID where it is located.

The interface features advanced filtering capabilities, allowing users to sort the list by crop type (e.g., cereals, legumes) or by growing season (e.g., Kharif, Rabi). This is particularly useful for generating crop recommendations based on historical performance and soil suitability. By maintaining an accurate inventory of what is growing where, AgroAssist helps prevent overlapping schedules and ensures that fertilizers and irrigation are applied precisely. Authorized users can quickly update a crop's status if growth is ahead of schedule or delete records of harvested plots to keep the active database clean and relevant for the current season.

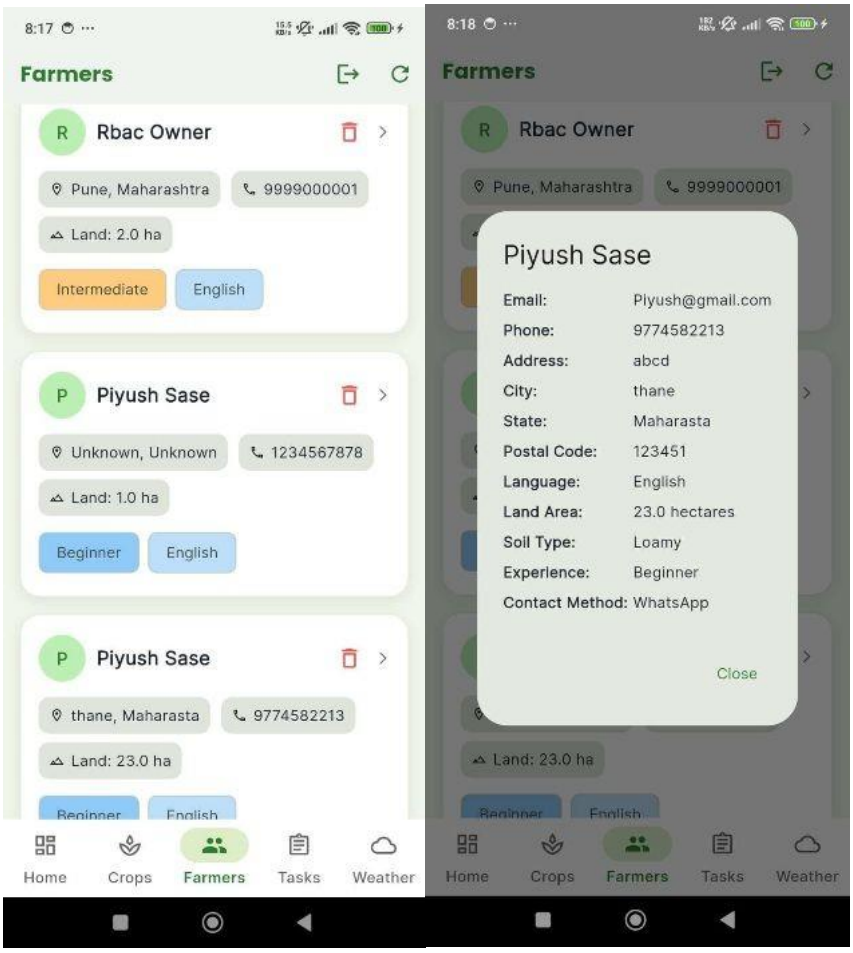


Figure 7.4: Farmer Info Screen

Figure 7.4 shows the Farmer Profile management screen. Admin users can view registered farmer profiles, update contact details, and link farmers to specific crops. Each farmer profile contains personal and farm-related information. focuses on the human element of agriculture through the **Farmer Profile Management** screen. This module functions as a specialized CRM (Customer Relationship Management) tool for the farm's workforce. Admin users can access a searchable list of all registered farmers, viewing detailed profiles that include contact information, emergency details, and specialized skill sets.

A unique feature of this screen is the ability to **digitally link** specific farmers to particular crops or field zones. This mapping ensures that the right expertise is applied to the right task—for example, assigning a farmer with experience in irrigation systems to a plot requiring complex water management. Each profile serves as a historical record of the farmer’s contributions, linked directly to the tasks they have completed. This level of organization not only aids in daily logistics but also provides valuable data for performance reviews and helps in distributing the workload equitably across the entire team.

Figure 7.5: Task Management Screen

Figure 7.5 demonstrates the Task Management module where tasks are created, assigned to farmers, and tracked through their lifecycle. The status field shows the current state (Pending / In Progress / Completed) and transitions are validated by the backend. demonstrates the **Task Management** module, which serves as the operational engine of the AgroAssist application. This screen is where abstract plans are converted into concrete actions. Admins can create new tasks, define their scope, and assign them directly to specific farmers. The UI prominently displays the **Task Lifecycle**, showing whether a job is 'Pending', 'In Progress', or 'Completed'.

To maintain data integrity, the system utilizes a strict state machine on the backend. Transitions are validated in real-time; for instance, a farmer cannot mark a task as 'Completed' unless they have first moved it to 'In Progress'. This ensures that the time-tracking and progress reports are accurate. The dashboard provides a progress bar for each task, allowing managers to see exactly how much work remains for any given project. By digitalizing the task-assignment process, AgroAssist replaces confusing verbal instructions or paper logs with a clear, auditable trail of accountability that ensures nothing falls through the cracks.

Figure 7.6: Weather Alerts Screen

Figure 7.6 presents the Weather Alerts screen which displays current weather advisories and forecasts relevant to the farm location. Alerts are stored in the backend and served to the Flutter frontend for timely display to farmers. presents the **Weather Alerts** screen, a vital safety and planning tool that bridges the gap between meteorological data and field operations. This module integrates with external weather APIs to fetch hyper-local forecasts tailored to the farm's specific GPS coordinates. The screen displays current conditions—such as temperature, humidity, and wind speed—alongside automated advisories and severe weather warnings.

These alerts are not just live feeds; the backend stores historical weather events to help correlate climate patterns with crop yields. When a significant weather event is predicted (such as a sudden frost or heavy downpour), the system can push notifications directly to the Flutter frontend, alerting farmers who are currently in the field. This timely display of information allows for proactive measures, such as covering sensitive crops or delaying pesticide applications that would otherwise be washed away. By making weather data central to the user experience, AgroAssist helps minimize environmental risks and maximizes the efficiency of every hour spent in the field. . When a significant weather event is predicted (such as a sudden frost or heavy downpour), the system can push notifications directly to the Flutter frontend, alerting farmers who are currently in the field. This timely

Chapter 8

Conclusion

AgroAssist has been successfully designed and developed as a full-stack agriculture management platform that addresses the real-world challenges faced by farmers and agricultural administrators. The system provides a unified, role-aware interface for managing crop data, farmer profiles, farm tasks, and weather alerts through a combination of a Django REST Framework backend and a Flutter frontend application.

The project demonstrated the effective use of modern software technologies including Python, Django, Dart, and Flutter in building a scalable and deployable agricultural platform. The implementation of token-based authentication ensured secure access control, while the role-based system successfully differentiated admin and farmer access levels. The task lifecycle management system with backend validation proved to be a robust solution for tracking and managing farm activities.

The project also provided valuable learning experience in REST API design, client-server integration, cross-platform mobile development, and agile project management. The team successfully overcame challenges related to API integration, session management, and Flutter web deployment during the development process.

In future, the system can be enhanced by integrating real-time satellite imagery for crop health monitoring, IoT-based soil sensors for automated data collection, AI-based crop yield prediction, and multilingual support to make the platform accessible to farmers across different linguistic regions of India. A production-grade PostgreSQL deployment with cloud hosting can further improve the platform's scalability and availability.

References

Django Software Foundation, "Django documentation," Django, 2024. [Online]. Available: https://docs.djangoproject.com/

Django REST Framework, "Django REST Framework documentation," 2024. [Online]. Available: https://docs.flutter.dev/

Google LLC, "Flutter documentation," Flutter, 2024. [Online].

Dart Team, "Dart programming language," dart.dev, 2024. [Online]. Available: https://github.com/Satyam-ptl/AgroAssist

GitHub, Inc., "AgroAssist Repository," GitHub, [Online]. Available:

SQLite Consortium, "SQLite documentation," SQLite, 2024. [Online]. Available: https://www.sqlite.org/docs.html

Vercel Inc., "Vercel deployment documentation," Vercel, 2024. [Online]. Available: https://vercel.com/docs

Fielding R. T., "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. Dissertation, University of California, Irvine, 2023.

Indian Council of Agricultural Research (ICAR), "Digital Agriculture in India," ICAR Report, Ministry of Agriculture, New Delhi, 2019.

Pressman, R. S., Software Engineering: A Practitioner's Approach, 8th ed., McGraw-Hill Education, New York, 2014.